

Polymorphism in Java

Before understanding polymorphism, first understand one important thing:

Why did we need Polymorphism?

Inheritance already allowed code reuse.

Example:

```
class Animal{  
    void sound(){  
        System.out.println("Animal makes sound");  
    }  
}
```

```
class Dog extends Animal{  
}
```

Dog got all properties from Animal.

So what problem still existed?

Problem with Only Inheritance

Suppose every child behaves differently.

A dog barks.

A cat meows.

A lion roars.

If all children use same parent method, behavior becomes wrong.

Example:

```
class Animal{  
    void sound(){  
        System.out.println("Animal sound");  
    }  
}
```

```
class Dog extends Animal{  
}
```

```
class Cat extends Animal{  
}
```

Output:

Animal sound
Animal sound

But real world behavior is different.

Dog should bark.
Cat should meow.

So child classes needed freedom to provide their own behavior.

That is where **Polymorphism** came.

What is Polymorphism?

The word comes from:

- Poly → Many
- Morph → Forms

Meaning:

"One thing behaving in many forms"

Real Time Example

Example 1 — Human Behavior

One person:

- At office → Employee
- At home → Father
- With friends → Friend

Same person.

Different behavior.

This is polymorphism.

Java Definition

Polymorphism means:

Same method name can perform different actions.

Types of Polymorphism in Java

1. Compile Time Polymorphism
(Method Overloading)
2. Run Time Polymorphism
(Method Overriding)

1. Compile Time Polymorphism

Method Overloading

What is Method Overloading?

Same method name
but different parameters.

Example

```
class Calculator{  
  
    void add(int a, int b){  
        System.out.println(a+b);  
    }  
  
    void add(int a, int b, int c){  
        System.out.println(a+b+c);  
    }  
}
```

Here:

`add()`

has multiple forms.

This is method overloading.

Why called Compile Time Polymorphism?

Because JVM/compiler decides during compilation which method to call.

Decision is made before program runs.

Rules of Method Overloading

Methods MUST differ by:

1. Number of parameters

`add(int a, int b)`

`add(int a, int b, int c)`

2. Type of parameters

`display(int a)`

`display(String a)`

3. Order of parameters

```
test(int a, String b)
```

```
test(String b, int a)
```

Invalid Overloading

Return type alone cannot overload

Wrong:

```
int add(int a, int b){  
    return a+b;  
}
```

```
double add(int a, int b){  
    return a+b;  
}
```

Reason:

Method signature is same.

Compiler gets confused.

Important Rules

Access modifier can change

Valid:

```
private void m1(){}
```

```
public void m1(int a){}
```

static methods can overload

```
static void show(){}
```

```
static void show(int a){}
```

Constructor can overload

```
class Test{
```

```
    Test(){}
```

```
    Test(int a){}
```

```
}
```

Real Time Example of Overloading

Mobile Camera App

Same button:

```
capture()
```

Different usage:

```
capture()
```

```
capture(portraitMode)
```

```
capture(nightMode)
```

```
capture(50MP)
```

Same action name.

Different parameters.

2. Run Time Polymorphism

Method Overriding

What is Method Overriding?

Child class provides its own implementation of parent method.

Example

```
class Animal{

    void sound(){
        System.out.println("Animal sound");
    }
}

class Dog extends Animal{

    void sound(){
        System.out.println("Dog barks");
    }
}
```


Main Method

```
public class Test{  
  
    public static void main(String[] args) {  
  
        Animal a = new Dog();  
  
        a.sound();  
    }  
}
```

Output:

Dog barks

Important Understanding

Reference type:

Animal

Object type:

Dog

JVM checks actual object during runtime.

Therefore:

Runtime Polymorphism

Why Overriding is Powerful?

Without overriding:

Every child behaves same.

With overriding:

Each child behaves differently.

Real World Example

Payment System

Parent:

```
class Payment{  
    void pay(){  
        System.out.println("Generic payment");  
    }  
}
```

Children:

```
class UPI extends Payment{  
    void pay(){  
        System.out.println("UPI payment");  
    }  
}
```

```
class CreditCard extends Payment{  
    void pay(){  
        System.out.println("Credit card payment");  
    }  
}
```

Now:

```
Payment p = new UPI();  
p.pay();
```

Later:

```
p = new CreditCard();  
p.pay();
```

Same method:

```
pay()
```

Different behavior.

Rules of Method Overriding

Rule 1 — Method signature must be same

Correct:

```
void show()
```

must override:

```
void show()
```

Rule 2 — Return type must be same or covariant

Valid:

```
Animal get(){  
}
```

Child:

```
Dog get(){  
}
```

Because Dog IS-A Animal.

Rule 3 — Access modifier cannot be more restrictive

Valid:

Parent:

```
protected void show()
```

Child:

```
public void show()
```

Invalid:

Parent:

```
public void show()
```

Child:

```
private void show()
```

Rule 4 — static methods cannot truly override

They are method hiding.

```
class A{  
    static void test(){}  
}
```

```
class B extends A{  
    static void test(){}  
}
```

This is NOT runtime polymorphism.

Rule 5 — final methods cannot override

```
final void show(){}  

```

Child cannot change it.

Rule 6 — private methods cannot override

Private methods belong only to that class.

Not inherited.

Rule 7 — Constructor cannot override

Constructors are not inherited.

Rule 8 — Exception Rules

Child can throw:

- same exception
- child exception

but not broader checked exception.

Important Internal Working

When you write:

```
Animal a = new Dog();
```

Java allows this because:

Dog IS-A Animal

At runtime JVM checks actual object.

If object is Dog:

```
Dog sound()
```

gets executed.

This is called:

Dynamic Method Dispatch

Most Important Interview Point

Overloading vs Overriding

Feature	Overloading	Overriding
Happens in	Same class	Parent-child class
Parameters	Must change	Must remain same
Return type	Can change	Same/covariant
Compile or runtime	Compile time	Runtime
Inheritance needed	No	Yes
Purpose	Multiple versions	Specialized behavior

Common Mistakes

Mistake 1

Thinking changing return type is overloading.

Wrong.

Mistake 2

Using different parameters in overriding.

Wrong.

Then it becomes overloading.

Mistake 3

Trying to override static method.

Not true overriding.

Mistake 4

Trying to reduce visibility.

Wrong:

public -> private

Important Scenario You Asked

"What if we override parent class and one subclass extends parent class?"

Example:


```
class Parent{  
  
    void property(){  
        System.out.println("Parent Property");  
    }  
}
```

```
class Child extends Parent{  
  
    void property(){  
        System.out.println("Child Property");  
    }  
}
```

Now:

```
Parent p = new Child();
```

```
p.property();
```

Output:

Child Property

Why?

Because actual object is:

```
new Child()
```

JVM gives priority to overridden child method.

Another Important Scenario

What if child does not override?

```
class Parent{  
  
    void show(){  
        System.out.println("Parent show");  
    }  
}  
  
class Child extends Parent{  
}
```

Now:

```
Parent p = new Child();
```

```
p.show();
```

Output:

Parent show

Because child did not provide its own version.

Multi-Level Overriding

```
class A{  
    void show(){  
        System.out.println("A");  
    }  
}  
  
class B extends A{
```

```
void show(){  
    System.out.println("B");  
}  
}
```

```
class C extends B{  
    void show(){  
        System.out.println("C");  
    }  
}
```

Now:

```
A obj = new C();
```

```
obj.show();
```

Output:

C

JVM always checks actual object.

Final Real Time Example

Food Delivery App

Parent:

```
class Delivery{  
    void deliver(){  
        System.out.println("Normal delivery");  
    }  
}
```

Children:

```
class BikeDelivery extends Delivery{
    void deliver(){
        System.out.println("Bike delivery");
    }
}
```

```
class DroneDelivery extends Delivery{
    void deliver(){
        System.out.println("Drone delivery");
    }
}
```

Now app can write generic code:

```
Delivery d;
```

```
d = new BikeDelivery();
d.deliver();
```

```
d = new DroneDelivery();
d.deliver();
```

Same method:

```
deliver()
```

Different behaviors.

This is the heart of polymorphism.

Final Conclusion

Polymorphism solved the biggest limitation of inheritance:

"Different child classes needed different behaviors."

Using polymorphism:

- same method name
- different behavior
- flexible design
- reusable code
- cleaner frameworks
- loose coupling

That is why polymorphism is one of the most powerful concepts in Java OOP.